

Keyless Image Signing & Attestation

How Sigstore, Fulcio, Rekor and GitHub OIDC sign and verify container images — end to end.

Platform Engineering Capstone · demo-api-go release pipeline & Ratify admission policy · July 2026

1 The core idea

Traditional signing relies on a **long-lived private key** that you must store, rotate, and protect — the same key whose corruption produced the `invalid pem block` error. **Keyless** signing removes that key entirely. Instead of "something you hold," trust is anchored to "**something you are**": GitHub proves the signer is the demo-api-go release workflow, a certificate authority issues a certificate valid for only ~10 minutes, the image is signed, and a public ledger records the event permanently. There is no durable secret to leak.

2 The components

Component	Role
GitHub OIDC provider <code>token.actions.githubusercontent.com</code>	Vouches for <i>who</i> is running the job — issues a signed JWT identity token bound to the exact workflow.
Fulcio (Sigstore CA)	Exchanges a valid OIDC token for a short-lived X.509 signing certificate that binds an ephemeral key to that identity.
Rekor (Sigstore)	Public, append-only, tamper-evident transparency log of every signature and attestation.
CT log	Transparency log of every certificate Fulcio issues — makes a misbehaving CA detectable.
cosign	The client that orchestrates token → cert → sign → log → store, and performs verification.
TUF root	The Update Framework metadata that tells cosign / Ratify the <i>real</i> public keys of Fulcio, Rekor and the CT log.

3 Signing flow (in the `release` job)

GitHub Actions runner	Fulcio	Rekor	GHCR
1. request OIDC token (aud=sigstore)			
<-- JWT (sub=...demo-api-go...) -			
2. generate EPHEMERAL keypair in RAM			
3. send pubkey + JWT ----->			
<-- short-lived CERT (SAN=identity, ~10 min) -			
4. sign image DIGEST with ephemeral privkey			
5. upload {signature, cert} ----->			
<----- Rekor entry + inclusion proof -----			
6. DISCARD the private key (gone forever)			
7. push signature as OCI artifact ----->			
		(...sha256-<digest>.sig)	

1. **Get an identity token.** Because the job declares `permissions: id-token: write`, cosign asks GitHub for an OIDC JWT (audience `sigstore`). Its claims include the identity `https://github.com/rkdutta/demo-api-go/.github/workflows/release.yml@refs/heads/main` and issuer `https://token.actions.githubusercontent.com`. Only your actively-running workflow can obtain this — it cannot be forged.
2. **Ephemeral key.** cosign generates a keypair in memory for this single run.
3. **Fulcio issues a certificate.** cosign sends the ephemeral public key plus the OIDC token to Fulcio. Fulcio validates the token against GitHub's issuer, then issues an X.509 certificate that **binds the ephemeral public key to your workflow identity** (identity in the Subject Alternative Name, issuer in an extension). Valid ~10 minutes. Fulcio also records the issuance in the CT log and embeds an SCT in the cert.
4. **Sign the digest.** cosign signs the image's `sha256:...` digest with the ephemeral private key.
5. **Record in Rekor.** cosign uploads `{signature, certificate}` to Rekor, which appends it to its public log and returns a **signed entry timestamp (SET)** — proof that this identity signed this digest at this moment.
6. **Discard the key.** The ephemeral private key is destroyed. There is no secret to rotate, leak, or manage.
7. **Store beside the image.** The signature — carrying the certificate and Rekor bundle in its annotations — is pushed to the registry as an OCI artifact tagged `sha256-<digest>.sig`, alongside the image.

4 Attestation flow (SBOM · vuln · quality)

An attestation uses the *identical* trust mechanism; only **what is signed** differs:

- A plain **signature** signs the bare image digest.
- An **attestation** signs a **predicate** — an `in-toto` Statement asserting `"image sha256:... has this SBOM / this vulnerability report / test coverage 96.5%."` It is wrapped in a DSSE envelope, signed through the same Fulcio-certificate + Rekor path, and stored as `sha256-<digest>.att`.

In the pipeline, `cosign sign` produces the `.sig`, and each `cosign attest --type cyclonedx | vuln | https://demo-api/quality` produces an `.att` carrying a `predicateType`. All four flow through steps 1–7 above.

5 What ends up in the registry

Artifact	OCI tag (derived from digest)	Contents
Image	<code>:0.0.0-abcdef → sha256:...</code>	The container itself
Signature	<code>sha256-<digest>.sig</code>	Signature + Fulcio cert + Rekor bundle
Attestations	<code>sha256-<digest>.att</code>	DSSE-wrapped SBOM, vuln, quality predicates

Everything is content-addressed to the image digest, so signatures and attestations always travel with the exact image they describe.

6 Verification flow

Two places run the *same* checks: the pipeline's **Verify (fail closed)** step immediately after signing, and the **cluster at admission time** (Gatekeeper calls Ratify). Given an image digest, the verifier:

1. **Fetches** the `.sig` / `.att` artifact from the registry.
2. **Validates the certificate** — checks it chains to the **Fulcio root CA** (known via the TUF trust root) and that its SCT proves CT-log inclusion.
3. **Checks the identity** — the certificate's SAN must match `--certificate-identity(-regexp)` and its issuer extension must match `--certificate-oidc-issuer`. **This is the actual trust decision.**
4. **Verifies the signature** over the digest using the public key in the certificate.
5. **Verifies the Rekor entry** — the signature is present in the transparency log, and Rekor's timestamp falls **within the certificate's ~10-minute validity window**. This is the key trick: the certificate expired long ago, but Rekor proves signing happened while it was valid — so short-lived certificates still verify indefinitely.
6. **(attestations)** additionally confirm the `predicateType` and that the in-toto statement's subject digest matches the image.

If any check fails, verification fails — the CI job fails, or the Pod is **denied at admission**.

7 Why the identity matters — the #1 gotcha

Fulcio will issue a certificate to **anyone** presenting a valid OIDC token — any GitHub repository on earth. So *"is there a valid Sigstore signature?"* is **not** a sufficient question; a signature alone only proves that *some* identity signed it. Security comes entirely from pinning **which** identity:

```
certificateIdentityRegExp: ^https://github\.com/rkdutta/demo-api-go/.github/workflows/release\.yaml@ref
certificateOIDCIssuer:     https://token.actions.githubusercontent.com
```

In plain words: admit an image only if it was signed by the **demo-api-go release workflow**, running on `main` or a `v*` tag, authenticated by GitHub's OIDC. Any other party's otherwise-valid Sigstore signature is rejected. This exact policy now lives in both the CI verify step and the cluster's Ratify configuration.

8 How it maps to this platform

demo-api-go CI (release.yml)

```
-----  
build image --> push to GHCR  
cosign sign  (steps 1-7) --> .sig in GHCR  
cosign attest x3          --> .att in GHCR  
cosign verify (identity) --> gate passes
```

kind cluster

```
-----  
Pod created in a tenant ns  
Gatekeeper webhook fires  
  |__ calls Ratify (external data)  
      |__ Ratify runs checks 1-6  
          against Fulcio / Rekor  
pass --> ADMIT  fail --> DENY
```

9 Operational requirements

1. **Ratify needs egress to public Sigstore infrastructure** — the Fulcio root via TUF (`sigstore.dev`) and Rekor (`rekor.sigstore.dev`). In a locked-down kind cluster this may be unreachable; keyless verification **fails closed** if it cannot reach Rekor.
2. **The signature must live in the registry the cluster pulls from.** Keyless signatures are OCI artifacts stored next to the image. CI signs in **GHCR**; if the image is mirrored into Harbor / the kind registry, copy the artifacts too:

```
cosign copy ghcr.io/rkdutta/demo-api@<digest>  
harbor.127.0.0.1.sslip.io/platform/demo-api@<digest>
```
3. **Legacy key-based tooling.** Any image still signed with the old cosign key (e.g. a local `build-and-attest.sh` run) will no longer verify under the keyless trust policy — migrate it to keyless or add the key back as a second trust policy.